Vision for Autonomous Robots

Lab Project 1: Image Calibration & Line Following

Lisann Becker (12558141)
Batu Helvacioğlu (14992760)
Matey Krastev (15258130)
Marina Orozco González (15214052)

1 Introduction

This lab project focuses on implementing image calibration and line following using the "Robotics Access for Everyone" (RAE) robot. The primary aim is to design a robust pipeline that leverages computer vision techniques to enable the RAE robot to detect and follow ceiling lines within the university lab environment. The assignment is divided into three key components: camera calibration, line detection, and line following. Camera calibration was the initial step, crucial for correcting lens distortion and improving the accuracy of subsequent image processing tasks. Line detection followed, where the goal was to identify the bright ceiling lines in the captured images using Canny edge detection and our own approach, contour analysis.

The RAE robot used in this project is a four-wheeled and 12 x 12 x 3.2 cm sized open-source mobile robot which is equipped with four stereo cameras 1. Throughout the experimentation, we primarily used the front-right camera for a consistent perspective of the ceiling lines.



Figure 1: Front view of RAE robot. For this project, we utilised the right-front stereo camera.

Primarily, the lines in the lab environment were four distinct ceiling LED lights which served as the targets for line detection and following 2. Most frames analysed in this report were captured during daylight hours, making the task more challenging due to increased reflections.

The final objective of the assignment was to integrate the results of line detection into a real-time control system, allowing the RAE robot to autonomously follow the detected line. This was achieved by designing a line following pipeline that continuously interprets the detected line's orientation to generate appropriate movement adjustments. The pipeline involves a feedback



Figure 2: Frame of reference of the ceiling lights in the lab.

loop between the visual detection process and the robot's motion control, ensuring that the robot can dynamically adjust its trajectory based on the visual input. This report details the implementation of each step, discusses the challenges encountered, and analyses the performance of the devised algorithms in achieving robust line following behaviour.

2 Section 1: Calibrating the Camera of a RAE Robot

Camera calibration is an essential process in computer vision and robotics that ensures accurate mapping of 3D world points to 2D image points. Calibration involves estimating the intrinsic and extrinsic parameters of the camera, which define how the camera captures the scene.

The relationship between a 3D point $\mathbf{X} = (x, y, z, 1)^T$ in the world and its corresponding 2D image point $\mathbf{x} = (u, v, 1)^T$ is expressed by the projection equation:

$$\mathbf{x} = \mathbf{K}[\mathbf{R}\ \mathbf{t}]\mathbf{X}$$

Expanded form:

$$w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha & s & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

In the intrinsic matrix $\mathbf{K}$, α and β are the focal lengths along the x and y axes, s is the skew coefficient (ideally zero), and (u_0, v_0) are the principal point coordinates. Ideally, $\alpha = \beta$ (indicating square pixels), and (u_0, v_0) should be at the image center. However, real-world cameras often deviate from these ideals due to manufacturing errors, such as lens imperfections and sensor misalignment.

These deviations can offset the principal point, cause unequal focal lengths, and introduce radial or tangential distortions, particularly in wide-angle lenses like those used on the RAE robot. Such distortions can warp the image, bending straight lines and affecting the accuracy of 3D projections onto 2D images. To correct these issues, camera calibration estimates the intrinsic matrix $\mathbf{K}$, the extrinsic parameters $\mathbf{R}$ and $\mathbf{t}$, and the distortion coefficients.

We use ROS2's `camera_calibration` and OpenCV's `cv.calibrateCamera` to correct these inaccuracies for the RAE robot's stereo camera and ultimately ensure accurate ceiling line detection and following.

2.1 Topic list for camera calibration

To launch the camera calibration in ROS2, we first had to install the package `ros-humble-camera-calibration` in our environment. In addition, we supply additional arguments to the previous function to adjust the calibration to the chess board we were using, resulting in the following call:

```
ros2 run camera_calibration cameracalibrator --size 5x8 --square 0.03
--ros-args -r image:=/rae/right/image_raw -p camera:=/rae/right/camera_info
```

Where `size` stands for the number of internal corners of the chess board, `square` is the side length of the squares in the chess board. The rest of arguments set the camera to be calibrated to the right front one.

2.2 Finish the camera calibration and report the parameters

The camera captures the 50 images that will be used in calibration. We visualize some of them in Figure 3.

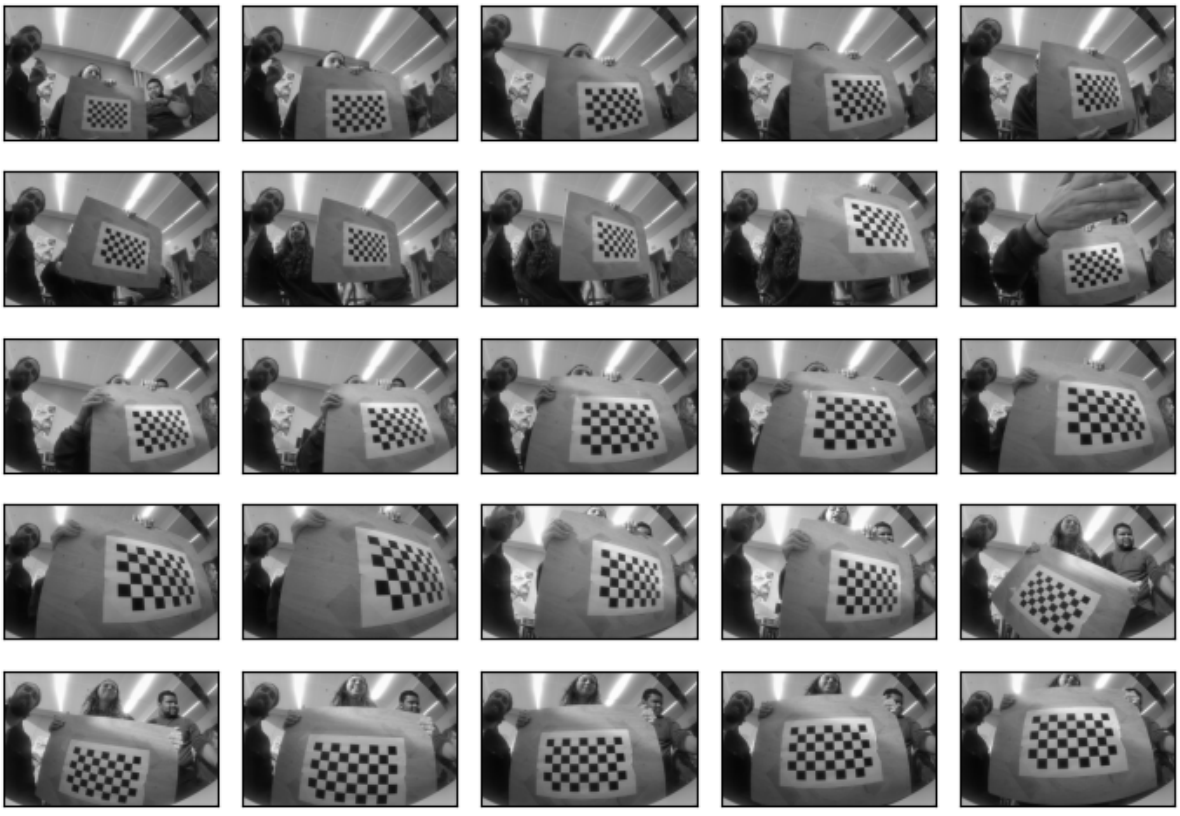


Figure 3: Some of the images that were captured for calibration.

We obtain the following intrinsic camera matrix from the `camera_calibration` package:

	k_1	k_2	k_3	p_1	p_2
ROS2: camera_calibrator	-0.260344	0.054780	0.000081	0.000311	0
OpenCV: CalibrateCamera	-0.2664491	0.08451640	-0.01322285	-0.0005280427	-0.001639067

Table 1: Distortion coefficients for ROS2 and OpenCV functions.

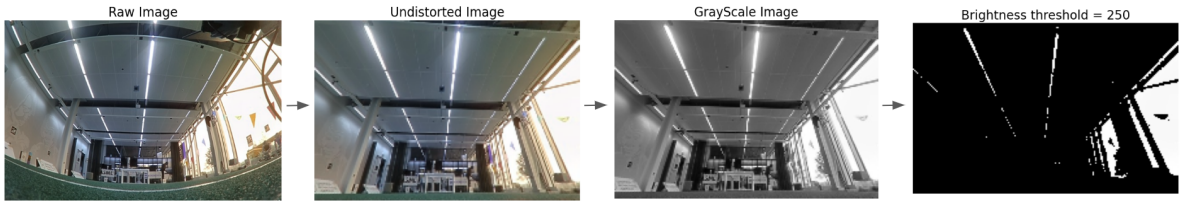


Figure 4: Pre-processing steps of the line detection pipeline.

$$\mathbf{K} = \begin{pmatrix} 290.298256 & 0.000000 & 317.439215 \\ 0.000000 & 290.215228 & 201.240386 \\ 0.000000 & 0.000000 & 1.000000 \end{pmatrix}$$

And the camera distortion coefficients $dist = \begin{pmatrix} k_1 & k_2 & k_3 & p_1 & p_2 \end{pmatrix}$ were:

$$\text{dist} = \begin{pmatrix} -0.260344 & 0.054780 & 0.000081 & 0.000311 & 0.000000 \end{pmatrix}$$

2.3 Do off-line calibration and reduce the distortion

In order to use an off-line more sophisticated calibration method, we used the `cv.calibrateCamera` method from OpenCV [1]. We choose this method due to its robustness and it's ease of use. We first use cv.findChessboardCorners [1] to extract the inner corners of the chessboard pattern. The 2D coordinates we extracted are used alongside 3D coordinates to calibrate the camera. The method returns the intrinsic camera matrix and the distortion coefficients. We use the camera matrix and distortion coefficients with cv.undistort [1] to get calibrated images.

The distortion coefficients $dist = \begin{pmatrix} k_1 & k_2 & p_1 & p_2 & k_3 \end{pmatrix}$ (note that p_1 and k_3 appear switched in the output compared to the function from ROS2) we got with this method are summarize in Table 1.

We can observe that the values we obtain for k_1 and k_2 are similar, while k_3 from OpenCV is 3-order-of-magnitude larger. k_3 is a higher-order radial distortion that is more subtle and affects the edges of the image especially in wide angle lenses. Since RAE robot uses a wide angle lens, we believe `cv.calibrateCamera` [1] is capturing our distortion better.

3 Section 2: Line Detection by a RAE robot

3.1 Design a line detection pipeline

In order to detect the lines that the lights in the lab form, we have developed a pipeline which first steps are visualized in Figure 4. These pre-processing steps are followed by a lines detector and a final step of filtering out unnecessary lines that we describe in the following sections.

At first, we undistort the image making use of the intrinsic matrix and distortion coefficients computed in the first section of the lab. As our interest lies on the lights, the key idea to be able to detect them is noting that a high brightness is what differentiates light from other objects



Figure 5: Bit-wise frames for different values of the brightness threshold.

in the scene. To capture this, we first convert the image to gray scale, as we won't need more information but brightness, and we create a bit-wise image by drawing a minimum threshold of brightness. We have explored the value that this threshold should have, as visualized in Figure 5.

We can see that the lowest the value of the threshold, more light reflections get into the masked image. However, if the threshold is too restrictive, as in the case of the $threshold = 254$, we might struggle capturing the form of the ceiling lights. For the rest of our experiments, we will set the brightness threshold to 245, as we have considered this value to be a trade off of both aspects.

Also, as noted in previous sections, a day-light scene represents a more challenging scenario due to the appearance of higher brightness in the windows of the lab and in the reflections. We will take this into account in the following steps: lines detection and lines filtering. These two last steps of our pipeline will be described in the following sections through two different versions of them: the first one, using Canny edge detector, and the second one being our own approach.

3.2 Realize pipeline with Canny edge detector

In the first approach for detecting lines, we use the traditional Canny edge detector [2] as a first step. In order to run it, however, we first implement a Gaussian blurred over the image, as it is usually perform for Canny edge detector to reduce noise. The detected edges pass through a Hough transform for detecting straight lines, were they are grouped in lines that adjust to them. This will leave us with several lines that don't necessarily correspond to the ceiling lights, as it can be seen in Figure 6, where the light of the windows creates most of the lines.

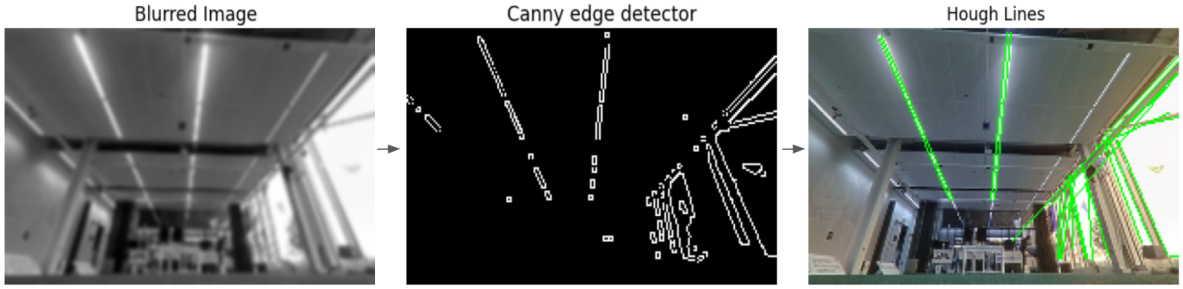


Figure 6: Steps for line detection using Canny Edge detector and Hough lines transform.

More examples of this algorithm performance can be seen in Figure 7.

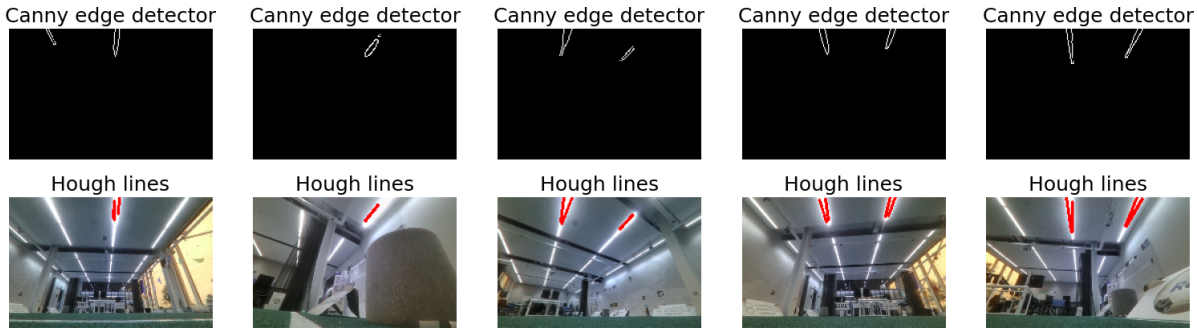


Figure 7: Canny edge detector and Hough lines for five different frames.

3.3 Explore and design more effective line detection algorithm

Now, let us introduce our alternative approach to the Canny edge detector. The previous method generated a large number of lines, most of which were not from the ceiling. The main motivation of our approach is to find the line corresponding to the closest light to the robot, by filtering out the rest of them. For this, we follow the following steps, which can also be visualized in Figure 8:

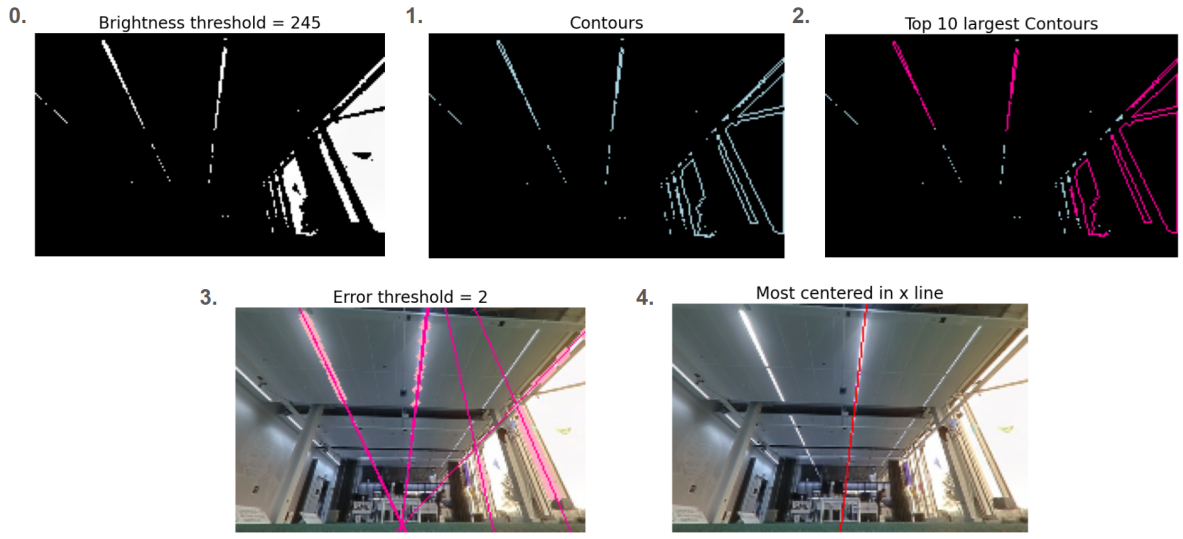


Figure 8: Step by step processing of the images for line detection with our approach.

1. Starting from the pre-processed images shown in Figure 4, which represents the binary image 0 in Figure 8, we now find the contours of the white regions making use of the `findContours` function from OpenCV [1].

2. From all the contours, we keep only the top 10 largest contours. With this, we aim to remove the noise in the frame.

3. Now, for each of the top 10 largest contours, we fit the points in the contours to a line through `fitLine` function from OpenCV [1], and we compute the error this regression through the mean of perpendicular distances from each point in the contour to the approximated line. We will filter out all those with an error bigger than 2. This value has been set empirically.

4. Finally, as we are interested in finding the line of the light in the ceiling that is closer to the robot, we take from the remaining lines, the one whose x coordinate for $y = 0$ is closer

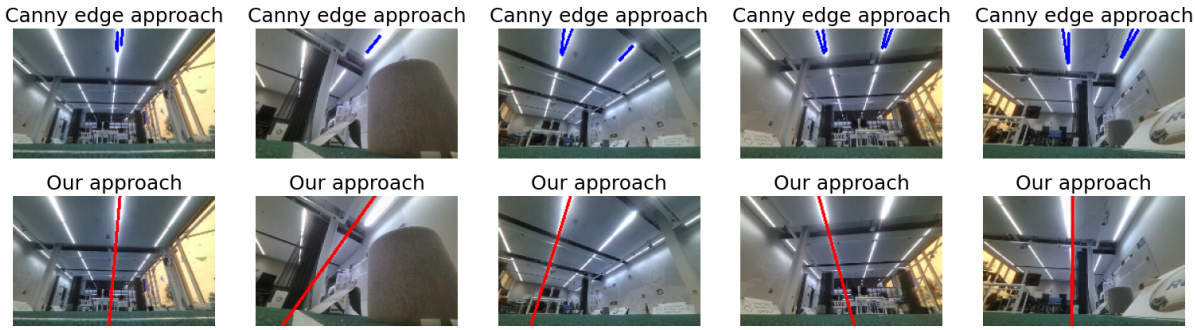


Figure 9: Comparison among Canny edge detections and our approach detections for five different frames.

to the middle of the image.

A comparison among detections with Canny edge approach with Hough lines and our approach can be seen in Figure 9. We can see that the main differences are that with the Canny edge approach we get more lines of less length than with our approach. Taking into account the next step of following a line, our approach will be more appropriate as that line will be what we based our control on. It is also worth noticing that images in Figure 9 have night light, which is an easier environment for detecting the ceiling lines, and both approaches seem to correctly detect the ceiling lights. However, if we compare Figures 6 and 8, where both approaches detect lines with day light, we notice that the amount of lines with the Canny edge approach is way higher than with our approach: in a difficult environment our approach works better.

4 Section 3: Line Following by a RAE robot

4.1 Design and run the line following pipeline

The final objective of this assignment was to enable the RAE robot to autonomously follow a line formed by the ceiling lights in the lab, making use of the line detection algorithm from previous sections and designing a line following pipeline that leverages real-time visual feedback to adjust the robot's movements and ensure it tracks the line throughout its trajectory.

As we previously advanced, we will make use of the closest line to the robot as reference, and our approach is based on trying to keep this line as vertical as possible. For this, we will pay attention to its slope: a high absolute value of this one indicates a vertical line. When this value decreases, it means that we are not straight following the line. From this we can get a lot of information, as the sign of the slope will also indicate us to which direction we should turn to correct the trajectory, as explained in the scheme of the example in Figure 10. In the caption of that figure we explained why the sign of the angular speed should be the same as the sign of the slope of the line we are following.

In a more technical way, we proceed as follows: After detecting the line, the **LineDetectionNode** publishes a directional message to the `LINE_DIRECTION_TOPIC`, containing the angular component (`angular.z`) of a `Twist` message. This angular value has been calculated as a proportional response to the line's deviation from the camera's vertical center, with positive values indicating a left turn and negative values indicating a right turn. The **LineFollowerNode** subscribes to this topic and interprets the direction message. It then calculates the velocity command for the robot: while it maintains a constant forward speed (`linear.x = 0.25`), the turning speed (`angular.z`) is modified using an arctan function ($0.3 * \frac{2}{\pi} * \arctan(\frac{1}{\text{slope}})$) to ensure that the sign of the angular speed will be the same as the one from the line's slope and that we

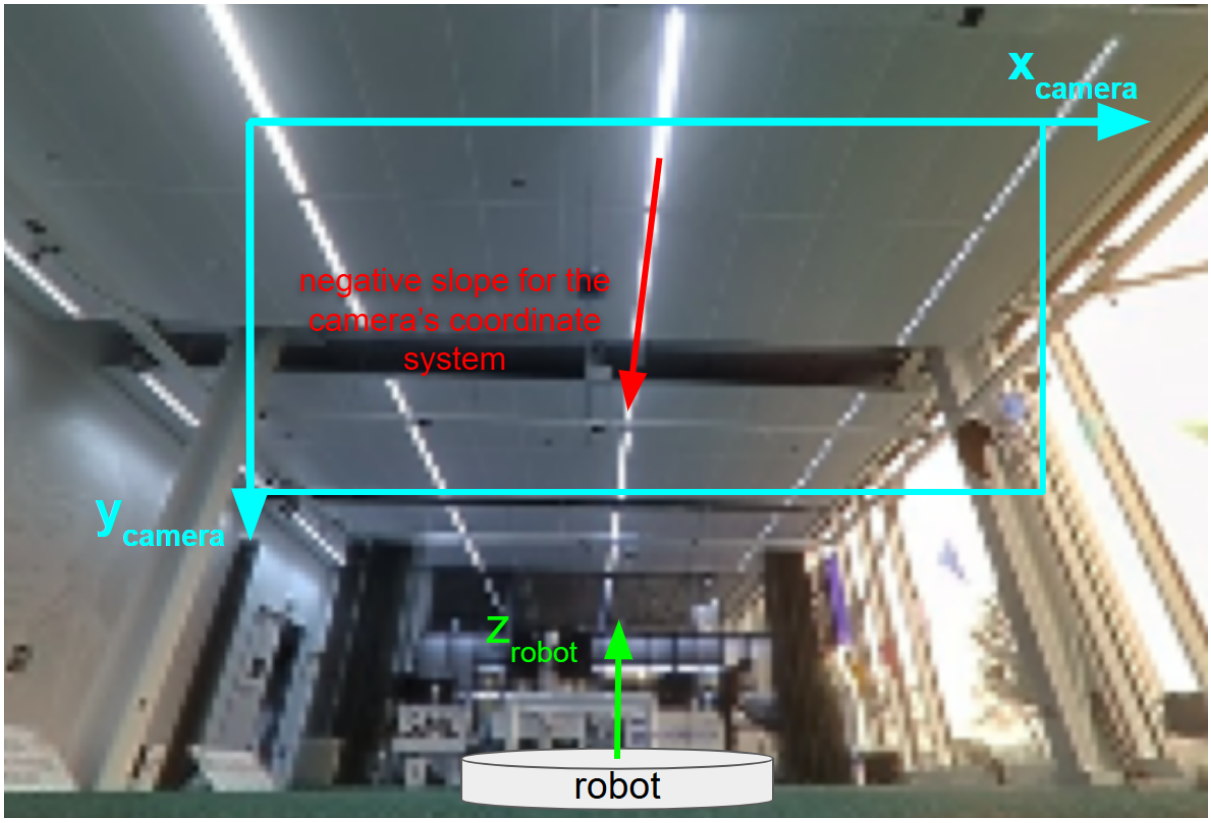


Figure 10: We can see in this picture a scheme of the coordinates systems of both the camera and the robot. In this example, we can see a negative slope, which means that we are moving to the left of the line (looking at the line from the robot's perspective), and that we need to turn right (negative angular speed for the z coordinate). In conclusion: the sign of the angular speed should be the same as the sign of the slope.

get smoother corrections for higher values of the slope (which means we are correctly following the line), and higher angular speed values for lower values of the slope (which means that we aren't following the line properly), while ensuring gradual adjustments. The value 0.3 has been chosen empirically as the maximum angular speed that works well, while $\frac{\pi}{2}$ is the maximum value that $\arctan(1/x)$ achieves in radians.

The calculated velocity command is published to the `/cmd_vel` topic, which directly controls the movements of the robot. This command includes both the forward speed and the angular turn, allowing the robot to continuously update its trajectory based on the line's orientation.

4.2 Keep following the same line as the RAE robot moves

As explained before, we keep following the same line as we correct the trajectory on each frame to keep having the followed line vertical and in the center of the image. As explained in the previous section, the sign of the angular speed switches from positive to negative according with the sign of the slope of the followed line, in order to keep it always centered. Also, the magnitude of the turn changes accordingly with the deviation of the absolute slope from being large.

Detecting a single line stably was a main challenge, especially given that the lab environment had varying light and reflection conditions. However, unlike methods that rely on a fixed number of lines (e.g., ceiling lights), our approach dynamically identifies the most central, vertical line without depending on specific features of the room the robot is located in. This is a main

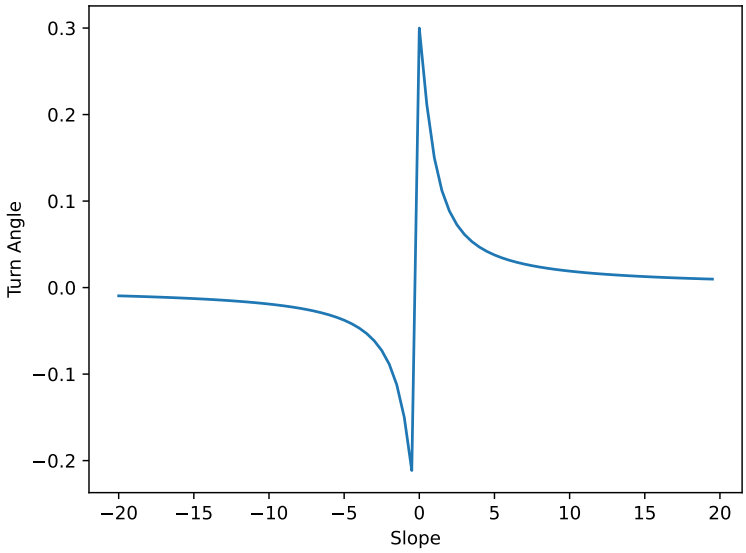


Figure 11: Function $\arctan(\frac{1}{x}) * 0.3 * \frac{2}{\pi}$

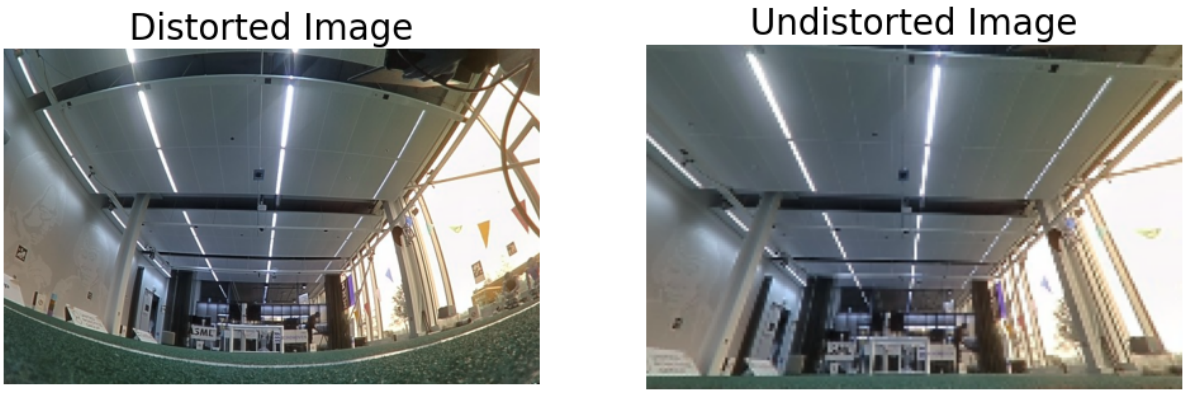


Figure 12: Distorted and undistorted images

advantage, as the RAE robot's line following can generalise effectively to any space, regardless of the number of lights.

4.3 Line following without camera calibration

We have tested the performance with our approach with and without camera calibration and the results were pretty similar. As we are focusing on the closest line, the one that is the most centered in the x axis, for being in the center it will suffer less distortion. We can visualize the effects of undistortion over the centered line in Figure 12.

However, when the line following starts failing and we deviate from the line, the effects of distortion are more notable, and the calculation of the slope and thus, of the angular speed, will be less precise.

4.4 Open Question: Solving remaining challenging problem

We decided to focus on the line in the center of the image as reference, as we thought that it would be easier to detect it through frames, while a pipeline based on multiple lines could fail more easily when not all lines are detected. Also, this approach reduces the probabilities of the robot to react to distracting elements in the frame. Of course, our approach also has limitations. For example, one of them is that we are taking the most centered line as the one that, for $y = 0$ has the most centered x. This might cause that, if there is a line with a big slope on the sides of the frame, it might get closer to the center than a vertical line that is slightly not centered. We could look for other ways of filtering those cases or of improving the track of lines.

During testing, we observed that while the RAE robot successfully detects the relevant line, its movement is often unstable as it swings left and right, and this affects the computation of the angular speed through the slope of the line. Initially, we designed a sinusoidal adjustment function to smoothen the robot's turning which is based on the line's deviation from the image center. However, this approach continued the behaviour when the deviation was large. To address this, we replaced the sinusoidal function with an arctan adjustment, which responds more proportional to the line offset and thereby increases stability.

We thought that the idea of moving such that the line keeps vertical would be easy to compute and intuitive. Theoretically, it should work even if the lines appeared perpendicular to the robot, as it should move to increase the slope of the line (this is, to make it vertical). In practice, this fails as this unstable scenario makes it difficult to keep track of the same line, as other lines appear more in the center than the one that you were trying to turn vertical.

5 Conclusion

In this lab project, we successfully implemented a pipeline for image calibration and line following on the RAE robot. By leveraging a contour-based line detection approach, where we kept track of the most centered line in the ceiling and making use of ROS commands for real-time feedback, we enhanced the robot's ability to detect and follow ceiling lines. Control in the robot's direction was implemented through an arctan-based angular adjustment while keeping the linear speed constant, and contributed to smoother and more stable line following.

References

[1] G. Bradski, A. Kaehler *et al.*, "Opencv: Open source computer vision library," https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html/, 2023, version 4.x, Camera Calibration and 3D Reconstruction module.

[2] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, no. 6, pp. 679–698, 1986.